

Medplum Overview and Fit for the OZRYN Project

What Medplum Is (and What It Is Not)

Medplum is an **open-source, “headless” EHR platform** built for developers. In essence, it provides a fully managed HL7 FHIR-compliant backend for healthcare applications ¹. It includes a secure clinical data repository with a FHIR-native data model, a REST/GraphQL API, built-in authentication/authorization, and comprehensive audit logging ². Medplum is designed as a **developer platform** – offering backend services (data storage, FHIR APIs, identity/auth) and tooling – rather than a turnkey clinical application for end-users. Its core is **FHIR-standard** (meaning all data is stored as FHIR resources by default ³), which makes it inherently interoperable and standards-compliant.

Importantly, Medplum is a **“headless” EHR**, meaning it does not enforce a specific user interface. Instead, it lets you build custom UIs or workflows on top of its backend services ⁴. Medplum **is not** a plug-and-play, low-code solution or a fully pre-built EMR product – you won't simply install it and get a complete clinical UI with all features configured. It's **developer-first** and requires a development team to customize and extend it for your specific needs ⁵ ⁶. In other words, Medplum provides the building blocks (FHIR data models, API, security, etc.) and example implementations, but **your team will design and build the actual application experience and any custom logic** on top of it. This flexibility is by design – it **avoids** the constraints of off-the-shelf EHRs but means Medplum itself doesn't inherently “do everything” without configuration. For example, Medplum's backend defines clinical data structures and workflows, but domain-specific intelligence or complex care pathways must be implemented by the developers (often using Medplum's extension points like FHIR extensions, bots, or custom code).

Boundary of scope: Medplum's out-of-the-box clinical model aligns with FHIR – it supports core resources for patient records (patients, practitioners, encounters, observations, orders, etc.), and it's extensible via FHIR **extensions/profiles** for custom data ⁷. It covers typical EHR backend needs (data storage, user/patient management, audit trails, access control) and provides **reference applications** (like a Provider Portal and Admin app) as starting points ⁴. However, it is **not** a full hospital information system or a one-size-fits-all clinical UI. For instance, Medplum has a basic **Provider App** (open-sourced) that you can use as a default EHR interface or fork for your own solution, but organizations often treat it as a **starting point or reference** rather than a final product ⁴. Medplum does **not** natively provide features outside the healthcare data domain (e.g. it isn't an analytics engine or AI platform on its own, though it can integrate with those). It's also not a replacement for robust BI/AI pipelines – rather, it ensures your health data is well-managed and compliant so that you *can* build analytics or intelligence on top of it. In summary, Medplum is the **backend “brain” and data spine** of an EHR system; it is not a ready-made patient-facing app or a no-code clinical tool – you will leverage its backend services and APIs to power the OZRYN system's own frontends and intelligence modules.

Key Medplum Features for OZRYN

For a project like **OZRYN** – which aims to unify medical records, behavioral signals, genetic data, etc., into a predictive/preventative health intelligence platform – Medplum offers several critical features and capabilities:

- **FHIR-Native Data Store & API:** Medplum's foundation is a FHIR-compliant datastore exposed via RESTful APIs (and GraphQL). This gives OZRYN a unified, **standards-based model** for all clinical and wellness data ². By using FHIR resources, OZRYN can represent diverse data (clinical observations, lab results, device readings, even genomics) in a structured way. Medplum supports the full CRUD operations and search on these resources out of the box ², including advanced querying (the FHIR search API and GraphQL for flexible queries ²). This FHIR backbone ensures **interoperability**: OZRYN can ingest or share data with external systems easily, since Medplum “speaks FHIR” natively ⁸. For example, if a hospital or wearable device provides FHIR or SMART-on-FHIR access, Medplum can directly consume that. (Medplum even notes that whenever possible, using FHIR/REST integrations is preferable to legacy interfaces ⁹.) The FHIR model also means OZRYN's analytics can leverage standardized codes and data formats (e.g. LOINC, SNOMED via Medplum's terminology services) – critical for making sense of medical and behavioral data. In short, Medplum would serve as **OZRYN's clinical data platform**, consolidating all patient information in a consistent format.
- **Patient Identity Resolution (EMPI):** Unifying records from multiple sources requires robust patient identity management. Medplum provides tools and patterns for an **Enterprise Master Patient Index (EMPI)** – in other words, deduplicating and linking patient records across systems. While not fully automatic, Medplum includes guides and reference implementations for **patient deduplication pipelines** ¹⁰. It describes how to ingest records from disparate systems, match potential duplicates, and merge them into a single master record ¹¹ ¹². The platform supports attaching multiple identifiers to a Patient (e.g., medical record numbers from different clinics, national IDs, etc.) and using those for matching ¹³. Through Medplum's **Subscription & Bot framework**, you can implement real-time duplicate checking: for example, set up a FHIR Subscription on new Patient creations and trigger a custom Bot function to check for matches and flag or merge duplicates ¹⁴. Medplum also logs all merges for auditability ¹⁵. In the **Medplum Enterprise** offering, there are out-of-the-box **EMPI features** (reports and workflows for deduplication) ¹⁶. For OZRYN, this means the platform can maintain a **single patient identity** even as data streams in from EHRs, wearables, and genomic databases, by using Medplum's patient matching/merging capabilities (with your custom rules). Effectively, Medplum can act as the **master index** of persons in OZRYN, ensuring all data about “John Doe” truly links to one unified profile (improving patient safety and data quality).
- **Clinical Data Ingestion & Interoperability:** Medplum is built to integrate with other health systems. Beyond FHIR APIs, it supports legacy healthcare interfaces like **HL7 v2 messages and C-CDA documents** for data exchange. Medplum includes an HL7 integration engine via its **Bots** and **On-Premise Agent**: you can configure Bots to consume or emit HL7 messages (ADT, ORU, SIU, etc.) as events ¹⁷ ¹⁸. For example, Medplum can listen for an ADT message (admission/discharge) from a hospital system and automatically create or update the corresponding Patient in its FHIR store ¹⁹. It also can generate outbound HL7 (e.g., send a lab order to a lab system in ORM format) ²⁰. The **Medplum Agent** is a small bridge you can self-host on a hospital network to securely ferry HL7/DICOM traffic between Medplum Cloud and on-prem devices ²¹ ²². In practice, OZRYN can leverage this to ingest data from *any* legacy EHR or lab system that only speaks HL7 – Medplum can

handle the translation into FHIR resources behind the scenes. Likewise, Medplum has pre-built integrations for common healthcare services: for example, **Health Gorilla** (lab orders/results), **DoseSpot** (e-prescribing), and connectivity to HIE networks like Carequality/CommonWell ²³ ²⁴ . Many of these are available in Medplum's higher-tier plans (Premium/Enterprise) or via APIs. With minimal configuration, OZRYN could use these connectors to pull in lab results or send electronic prescriptions through Medplum's unified API. In summary, Medplum gives OZRYN a robust **interoperability layer** – whether through FHIR, HL7, or third-party services – to ingest clinical data from varied sources and to share data out. This will be crucial for aggregating the “human data” OZRYN needs (medical records, wearables, genomics, etc.) into one system.

- **Audit Logging and Compliance:** Medplum was built with healthcare compliance (HIPAA, GDPR, etc.) in mind. Every interaction with the data can be recorded as an **AuditEvent** in the FHIR store ²⁵ , providing a detailed audit trail of who accessed or changed what. The platform ships with **comprehensive audit logging and activity tracking** enabled by default ² . This is invaluable for regulated environments: any access to patient data, any modification, can be tracked for security and compliance purposes. Moreover, Medplum's cloud offering is **HIPAA-ready and SOC 2 Type II certified**, and it achieved full **ONC Health IT certification** for an EHR platform ²⁶ . That means it meets strict requirements (like audit trails, role-based access, data export capabilities, etc.) out of the box. OZRYN can take advantage of these features to ensure **every data access is logged** and to simplify passing security audits. For example, Medplum's audit logs can be streamed into your observability tools (Sumo Logic, Datadog, Splunk) or a data warehouse ²⁷ ²⁸ for real-time monitoring. It also supports audit-specific tools – the enterprise “CISO Dashboard” aggregates security metrics and alerts ²⁹ ³⁰ . In practical terms, using Medplum as the data layer means OZRYN automatically gains an **audit trail for all key actions** on health data ³¹ . This frees your team from implementing a separate auditing system and helps maintain compliance (HIPAA, etc.) by default. The built-in **access controls** (down to resource-level policies) further ensure that only authorized roles can see or edit sensitive data ³² , which is critical when combining behavioral, genetic, and medical data in one platform.
- **Identity, Authentication, and Access Control:** Medplum includes a flexible identity and auth system tailored for healthcare scenarios. Users in Medplum can authenticate via built-in password, OAuth2/OIDC providers, SSO integrations (Google, Okta, Azure AD, etc.) ³³ ³⁴ , or even via external IdPs like Auth0 or Cognito ³⁵ . It supports **multi-tenant projects** and **user-role profiles**: e.g. a user can have a *Practitioner* profile to access multiple patients or a *Patient* profile to only access their own data ³⁶ . Fine-grained **Access Policies** can be configured to restrict which FHIR resource types each role can read/write ³⁷ ³⁸ . For OZRYN, this means you can readily implement scenarios like patient portals (patients only see their data), provider portals (doctors see their cohort), or research access (de-identified data) using Medplum's existing role-based model. The platform also supports **SCIM** for enterprise user provisioning ³³ and **audit of authentication events**. All these identity features will help OZRYN manage user accounts and permissions securely and in compliance (e.g., enforcing SSO or 2FA for clinicians if needed ³⁹). And because identities can be linked to Patient records, granting a patient user access to their own FHIR record is straightforward ⁴⁰ . This will be useful when OZRYN provides users insights or dashboards from their data – Medplum can serve as the secure gateway for that data access.
- **Workflow Automation (Bots & Subscriptions):** Medplum's **Bots** (serverless functions) and **Subscriptions** (event triggers) allow OZRYN to embed intelligence and automation in the data layer.

For example, you can deploy a JavaScript/TypeScript “Bot” in Medplum that runs whenever a certain event occurs (new data received, a threshold exceeded, etc.) ¹⁴ . This could power real-time health interventions: imagine OZRYN wants to flag a high-risk pattern – you could set a Subscription on relevant Observations (e.g., a spike in blood pressure or a critical lab value) and trigger a Bot to perform an action (like send an alert, create a Task for follow-up, or even call an external AI service). Medplum bots run within the Medplum environment (no extra servers needed) and can call external APIs or use Medplum’s API. This is highly relevant to OZRYN’s goal of **predictive/preventative intelligence** – you can codify certain predictive rules directly as bots. For instance, an “AI Scribe” bot could listen for new clinical notes and use an AI API to structure or summarize them (Medplum has demonstrated integrations with OpenAI in workflows ⁴¹). Or a “Care Gap” bot could periodically scan patient data for any preventive care needed. Essentially, Medplum gives you a **framework for custom workflows** on top of the data – **without deploying separate infrastructure**. OZRYN can leverage this for things like automating data ingestion (e.g., a bot to fetch wearable data daily), performing data normalization, or triggering personalized health insights when certain conditions are met. All such actions are logged (Medplum will record bot executions and any data changes they make in audit logs ⁴²), maintaining traceability. This capability ensures that **OZRYN’s intelligent logic can be tightly integrated with the data repository**, reacting immediately to new information.

In summary, Medplum brings a comprehensive feature set that OZRYN should capitalize on: a **standardized FHIR data model**, strong **patient identity management**, easy **data ingestion from varied sources (FHIR, HL7, etc.)**, built-in **audit/compliance infrastructure**, and broad **interoperability** tooling. These features will enable OZRYN to gather and manage the full spectrum of health data in a unified way and provide a rock-solid, compliant backbone for any analytics or AI built on top.

Out-of-the-Box Functionality vs. Required Customization

One of Medplum’s strengths is providing **robust defaults** for common needs, while also allowing deep customization for custom requirements. It’s important to delineate which capabilities work “**out of the box**” (with minimal configuration) and which require **custom development or configuration** in Medplum:

- **Available Out of the Box:** If you deploy Medplum (or sign up for Medplum Cloud), you get a working FHIR server with all standard resource types and a default relational schema. **User management** and **auth** are ready immediately – you can register users, assign them roles (Patient/Practitioner/etc.), and Medplum handles login, token issuance, and basic access control without extra coding ⁴³ ³⁶ . The platform automatically provides **audit trails** for data access and modifications (no need to build logging – it’s on by default) ² . Furthermore, Medplum includes **administrative and clinical reference apps** that run on day one. For example, by simply creating a Medplum account and a Project, you can log into the **Medplum App** (admin console) to view or manage data ⁴⁴ . Likewise, you can access a pre-built **Provider Portal** UI (at `provider.medplum.com`) which is an example EHR interface tied to your project’s data ⁴⁵ . This provider app includes basic features like patient registration, scheduling, charting of encounters, etc., once you load or input some data ⁴⁶ ⁴⁷ . These out-of-the-box apps mean you don’t have to start from scratch on the frontend – you have a working environment to explore your data and even use as a stopgap UI. Medplum also comes with **ready-made integrations** in higher-tier cloud plans: e.g. the Premium Cloud tier unlocks one-click use of lab ordering via Health Gorilla, or e-prescribing via DoseSpot, without you having to build those connectors ⁴⁸ ⁴⁹ . Additionally, core compliance

features (HIPAA safeguards, ONC certified functionality like audit logs, export formats, etc.) are **built-in** and require only configuration (for example, enabling “strict mode” on a Project to enforce stricter FHIR compliance is a simple flag). Summarily, Medplum’s **out-of-the-box** offering includes the fundamental **FHIR datastore** + API, security/compliance guardrails, a library of **React UI components** (and basic UIs), and support for common healthcare workflows (questionnaires, scheduling, messaging) through standardized FHIR resources and sample implementations.

- **Requires Customization or Development:** While the basics are provided, advanced or project-specific functionality will require writing code or config on top of Medplum. For instance, **Patient identity resolution (duplicate merging)** is not an automatic feature – you need to implement the matching logic (Medplum provides sample code and guides). As noted, you might set up a Bot to evaluate new Patient entries and merge duplicates based on your criteria ¹⁴. Medplum gives you the hooks (Subscriptions, Bot APIs) and even a reference “dedup bot” implementation ⁵⁰, but you must customize it to your context (deciding matching rules, manual review steps, etc.). **HL7 v2 ingestion** similarly requires some setup: out-of-the-box, Medplum doesn’t immediately start consuming HL7 feeds until you configure a Bot to parse messages and an Agent or interface to receive them ¹⁷. The HL7 interface engine Medplum provides is highly **customizable**, meaning you’ll write scripts to map HL7 segments to FHIR resources (Medplum offers guidance and sample bots for ADT, lab results, etc., but you’ll adapt them to your sources) ⁵¹. Another area is **UI/Workflow customization**: The default provider portal is generic – if OZRYN has unique workflows (say, a genetic risk dashboard or a behavioral coaching module), you’d likely **build custom front-end components** using Medplum’s React component library and APIs ⁵² ⁵³. Medplum’s **UI kit** covers basics (login screens, patient chart components, forms) which accelerates development, but the **specific user experience is up to the developer** ⁵⁴ ⁵⁵. In short, any tailored user-facing functionality (e.g. a specialized patient engagement app) will involve coding, albeit on top of Medplum’s framework. Other examples of required customization include: defining **custom FHIR extensions** if you need to store data that doesn’t fit the standard resource fields (Medplum fully supports FHIR extensions, but you define and use them in your app logic) ⁷; setting up **business logic bots** (e.g. a preventive care reminder bot – you’d write the logic using Medplum’s SDK); and integrating external analytics or AI – which you’d do by calling external APIs from a bot or by exporting data to your own services. Essentially, **Medplum provides the platform primitives** (data models, rules engine, integration hooks), but **OZRYN’s unique features will be implemented by configuring those primitives or adding code**. This aligns with Medplum’s positioning: it’s not a low-code solution with pre-built bespoke workflows, but rather a **flexible toolkit**. The Medplum team explicitly notes that success with the platform “depends on having the right technical team” who can build on Medplum’s foundation ⁵. Conversely, features like **lab integration and eRx** that Medplum Cloud offers in Premium could be seen as “out-of-the-box” if you use that service – but if you self-host, achieving those will require integrating with lab APIs yourself or using Medplum’s open connectors with proper credentials. To summarize, **basic EHR capabilities are turnkey**, but **project-specific capabilities (identity matching, custom AI, novel data types, unique workflows)** will require development effort using Medplum’s tools. The good news is Medplum’s openness and modularity make such customization feasible – you’re not locked out from extending it. You should plan on allocating developer time to leverage Medplum’s API, write bots, and customize UIs to fully realize OZRYN’s vision.

Medplum Cloud (PaaS) vs. Self-Hosting Components

Medplum can be adopted either as a **hosted service** (Medplum Cloud) or deployed in your own infrastructure. It's fully open-source (Apache 2.0), meaning all core components can be self-hosted if desired ⁵⁶ ⁵⁷. Here we clarify which parts of the system you might self-host versus use as a managed Platform-as-a-Service:

- **FHIR Server & Backend Services:** The Medplum **server (FHIR API backend)** is open source and can certainly be self-hosted. Medplum provides deployment templates for AWS, GCP, Azure, Docker, etc., to spin up the full stack in your environment ⁵⁸ ⁵⁹. Self-hosting means you run the Postgres database, the Node.js server application, and associated services within your own cloud or data center. This gives you full control (and responsibility) over the data and infrastructure. On the other hand, **Medplum Cloud** (the hosted service at `api.medplum.com` and `app.medplum.com`) is maintained by the Medplum team and offered in tiers (Free, Production, Premium, Enterprise) ⁶⁰. In Medplum Cloud, they handle the ops – database scaling, updates, security patches, backups, etc. – and you simply consume the API as a service. The functionality of the core FHIR API is the *same* in both cases (the cloud mirrors the open-source core ⁶¹), but the cloud version comes with certain managed integrations and certifications (for example, the Medplum Cloud is ONC-certified as a whole). **Which to choose?** For a quick start or small-scale usage, the cloud free tier is attractive – you can be up and running in minutes. For OZRYN, if you require **data residency control, custom infrastructure, or want to avoid SaaS dependency**, you can opt to self-host the Medplum server. Many teams start on the hosted version and later migrate to self-hosting once they need more control (Medplum provides a data export so you can migrate projects). There is also a middle ground: Medplum offers a **Self-Hosted Enterprise** plan where you run it in your cloud but with support from Medplum (and features like enterprise integrations) ⁶².
- **Medplum CLI and SDK:** The Medplum CLI (`npm init medplum`) is a developer tool to scaffold projects or manage deployments ⁶³. This is run locally by developers and is inherently self-hosted (there's no "cloud CLI"; you use it on your machine or CI pipeline). It helps generate boilerplate code, run a local dev server, or perform admin tasks via command line ⁶⁴. Likewise, the **TypeScript/JavaScript SDK** and React component library are packages you include in your apps – these are used in your development environment regardless of backend hosting. So, all **development tooling** (CLI, SDK, component library) is meant for you to use and host in your codebase; Medplum Cloud doesn't change that (except that it provides the backend endpoint to talk to).
- **Application Frontends (Medplum App, Provider/Patient Portals):** Medplum publishes its reference web applications as open source, which means you can deploy them yourself (and customize them). The **Medplum App** (the admin/developer console at `app.medplum.com`) and the **Provider Portal** (`provider.medplum.com`) are both available in the GitHub repo ⁶⁵ ⁴. If you self-host the backend, you would also self-host these UIs (so they point to your server's API endpoint). Medplum provides instructions for hosting the entire stack on various platforms ⁶⁶ ⁶⁷, and even a one-command Docker setup for development. In Medplum's cloud service, they host those UIs for you on their domain – when you use Medplum Cloud, you can simply log into their hosted app UI. You also have the option in cloud to **white-label** the API and app under a custom domain (some paid plans allow custom domain mapping) ⁴⁸. For OZRYN, if you want a completely integrated experience, you might fork the Provider Portal and rebrand/modify it, then host it at (for example) `ehr.ozryn.com` hitting either Medplum Cloud's API or your own instance. Similarly,

Medplum's **Patient Portal** sample (if provided, or created by you using their components) could be self-hosted. In short, **all front-end components can be self-hosted and tailored**, since you have the source; using Medplum Cloud doesn't stop you from doing this – you can still point your custom UI to the cloud API. Medplum's flexible approach means you're not locked into their hosted UI at all ⁶¹.

- **Managed Services vs Self-Managed Add-ons:** Some features are provided as **managed services** in Medplum Cloud's higher tiers which you'd otherwise need to manage or implement yourself when self-hosting. For example, **email/SMS sending** for notifications: Medplum Cloud Premium includes a certain volume of transactional emails (for invites, notifications, etc.) sent through their infrastructure, whereas a self-hosted deployment requires you to configure an SMTP or email gateway (Medplum has guides for integrating SendGrid, Mailtrap, etc.) ⁶⁸ ⁶⁹. Another example is the **terminology service**: Medplum Cloud comes pre-loaded with UMLS licensing for standard code sets (as indicated by "UMLS Terminology ✓" in pricing) ⁴⁸, so things like SNOMED or LOINC validation might work out of the box. If you self-host, you might need to load those code systems yourself or ensure you have a license. **Audit log streaming** to external systems is possible in both, but in cloud you toggle a setting to send logs to, say, AWS Athena or Datadog ²⁷, whereas self-hosted you'd configure those pipelines (Medplum provides OpenTelemetry support for self-hosted monitoring ⁷⁰ ⁶⁴). Finally, **enterprise integrations** (lab network access, HIE connectivity, EHR bridges) are offered as part of Medplum's enterprise/cloud services ²³ ⁷¹. If self-hosting, you can still integrate with labs or HIEs, but you'll be using Medplum's integration code with your own accounts/credentials or writing custom logic. Nothing is strictly "cloud-only" in terms of capability – the open-source repo contains the integration code paths (like support for C-CDA, SMART on FHIR, HL7, etc.) – but the difference is **who manages the relationship and infrastructure** for those services. Medplum Cloud will manage (and charge for) things like an off-site audit archive or a plug-in to Epic, whereas self-hosted means you operate those connections.

In summary, **Medplum's core can be self-hosted entirely**, and many users do this to retain data control. The **hosted Medplum Cloud** is essentially the same software delivered as a service, plus convenience and certain value-add services (with an SLA and support). For a large-scale initiative like OZRYN, you might start on Medplum Cloud (to quickly get infrastructure and perhaps certified capabilities running), and later consider self-hosting for customization, cost control, or integration into a larger private cloud environment. Medplum explicitly provides a "*self-hosting vs. cloud*" guide to help teams decide ⁷². Key components like the CLI, SDK, and UI libraries are always in your hands, whereas the choice for the **FHIR server** and related backend services comes down to build vs buy: run it yourself or have Medplum operate it. Either way, you have the freedom to migrate, since there's no proprietary lock-in (the data is standard FHIR and the code is open source ⁵⁶). Many teams use the managed service to **avoid DevOps burden** and ensure up-to-date compliance, unless they have requirements to host internally ⁵⁷ ⁷³.

Incorporating Medplum into the OZRYN Architecture

Given OZRYN's goal of a **unified, intelligent health data architecture**, Medplum would serve as the **central EHR/clinical data layer** within that system. Here's how you can architect and integrate it for maximum benefit:

- **Use Medplum as the Source of Truth for Clinical & Wellness Data:** All medical records, clinical observations, and relevant behavioral or genetic data should reside (or be mirrored) in Medplum's

FHIR datastore. In practice, OZRYN can funnel data from various inputs into Medplum: e.g. connect to hospital EHRs via FHIR API or HL7 feeds (using Medplum's integration engine) to ingest clinical records; receive wearable or sensor data through a custom API that converts them to FHIR Observation resources; store genomic test results as FHIR DiagnosticReport or Observation (genetic variant reporting is supported via FHIR extensions). By normalizing disparate data into FHIR and storing it in Medplum, you achieve a **single, standardized data model** across the entire OZRYN platform. This simplifies any downstream analytics because everything is in a consistent format with standard codes. Medplum's support for **USCDI and common terminologies** helps ensure, for instance, a lab code from one source is understood the same way in OZRYN ⁷⁴ ⁷⁵ . All data writes and reads go through Medplum's API, which enforces validation and access rules – this guarantees data integrity and security at the core. Essentially, Medplum becomes the **health record backbone** of OZRYN, analogous to an EHR database, but one that is fully interoperable.

- **Implement a Unified Patient Index and Data Aggregation Pipeline:** If OZRYN aggregates data from multiple organizations or data silos, plan on utilizing Medplum's EMPI capabilities. You might create a **data ingestion service** in OZRYN that receives incoming patient records from various sources, uses Medplum to either find a matching patient (via query on identifiers/demographics) or create a new one, and if needed, triggers Medplum's deduplication bot to merge duplicates ¹⁰ ¹² . This pipeline could be batch or real-time. Medplum's guidance on **Patient Deduplication Architectures** can inform your design – for example, decide on a strategy for matching (deterministic vs probabilistic) and whether merges happen automatically or with human review ⁷⁶ ⁷⁷ . By handling this within Medplum, you ensure that OZRYN's predictive algorithms always work on consolidated, complete patient records (preventing, say, one person being split into two records that confuse risk scoring). You can use Medplum's **audit reports** to trace any merges (so that data lineage is preserved, which is important for trust). In an architecture diagram, Medplum sits in the middle: multiple source systems feed into it, and it maintains the linkages and unified records.
- **Leverage Medplum for Interoperability and Data Exchange:** OZRYN will likely need to both **ingest and output** data to the broader healthcare ecosystem. Medplum can be your **interface layer** for this. For ingestion, use Medplum's integrations: e.g., set up the Medplum HL7 Agent at a partner clinic to push ADT feeds into OZRYN ²¹ ; connect Medplum to lab providers (Labcorp, Quest) via the built-in lab integration so that any lab test ordered in OZRYN (through Medplum's order entry) returns results into OZRYN automatically ⁷⁸ ⁷⁹ . For output/sharing, Medplum supports standards like **SMART on FHIR** and the upcoming TECCA/UDAP profiles – meaning you can allow external apps or health information exchanges to query OZRYN's data through Medplum's FHIR API with the proper OAuth2 scopes ⁸⁰ ⁸¹ . For example, if a patient wants to share their OZRYN record with a new doctor's EHR, Medplum's ONC certification ensures it can handle the required **Patient Access API** and Bulk Data export formats ²⁶ . You can also generate CDA documents or other formats from Medplum if needed for compatibility ⁸² . **Interoperability is critical** for an architecture dealing with comprehensive data, and Medplum essentially provides OZRYN with a ready-made interoperability module (FHIR endpoints, HL7 ingestion, HIE connectivity, etc.). This saves OZRYN from building custom interfaces for each external system – instead, connect them all to Medplum, which acts as the hub.
- **Incorporate OZRYN's Intelligent Modules via Medplum APIs and Events:** With Medplum managing the data, OZRYN's "intelligence" layer (predictive modeling, AI analytics, preventive algorithms) can be built on top of it. There are a few integration patterns to consider:

- *Batch analytics:* You might periodically export data from Medplum to a data lake or warehouse for heavy ML processing. Medplum Enterprise facilitates syncing data to analytics platforms like Snowflake or Redshift ⁸³ – you could use their log streaming or a direct database replica to keep your analytical store updated with new clinical data. This lets OZRYN run, say, a monthly risk stratification model on the latest unified dataset.
- *Real-time/event-driven logic:* For immediate preventive actions, use **Medplum Subscriptions** on relevant data changes. For example, when a new lab result comes in, a Subscription could notify an OZRYN microservice or trigger a Medplum Bot that scores the result using an AI model (this bot could call a deployed ML service or use an embedded model). Medplum's eventing ensures **nothing falls through the cracks** – every new piece of data can provoke an automated response if desired ¹⁴. OZRYN can thus embed real-time intelligence directly in the workflow (e.g., automatically alert a care coach if a patient's glucose readings trend high for a week, via a bot that watches Observation values).
- *Embedded AI in documentation:* Medplum doesn't do AI itself, but it's built to incorporate AI tools. A concrete example is using the **OpenAI integration** (Medplum can integrate with OpenAI's API as a partner, as listed in enterprise features ⁴¹) to do things like NLP on clinical notes or draft patient communication. In OZRYN, you might have a Bot that, whenever a clinician finishes writing a visit note (FHIR **DocumentReference** or **Observation**), sends it to an NLP service to extract structured data or identify care gaps, then saves those insights back via Medplum's API. This kind of pipeline is enabled by Medplum's combination of **Bots + external API calls** and the fact that Medplum's data is structured (FHIR), which an AI can more easily consume.
- *SMART on FHIR apps:* If OZRYN intends to include third-party health apps or allow external apps to run on its data, Medplum's support for **SMART App Launch** protocols is key. Medplum can function as an OAuth2 provider for SMART apps ⁸². Thus, OZRYN could integrate third-party modules (for example, a specialist decision support app) that users launch within OZRYN – the app would securely pull data from Medplum's FHIR API under the hood. This provides architectural flexibility: you don't have to build every feature in-house if a SMART app exists.
- **Self-Hosting Medplum as Part of OZRYN's Stack:** If OZRYN opts to self-host Medplum, it should be treated as a core microservice (or set of services) in the architecture. You'd deploy the Medplum server (potentially in a container or cluster), alongside OZRYN's other services (like an AI service, a user-facing web app, etc.). Make sure to follow Medplum's **reference architecture** for production – e.g., using AWS CDK templates for a scalable setup with load balancers and a managed database ⁶⁶. This ensures high availability for the FHIR API. Also, configure Medplum's environment for your needs: enable the **On-Prem Agent** where needed (for site-specific data exchange), set up **CORS** and custom domains so that OZRYN's front-end can call the API seamlessly ⁸⁴, and plug in your logging/monitoring (Medplum supports OpenTelemetry and can feed into your centralized monitoring system ⁷⁰). Essentially, Medplum would be one of the core services in the OZRYN Kubernetes cluster or cloud stack, with well-defined APIs that other OZRYN components consume. The **security model** should also be planned: use Medplum's OAuth for authenticating OZRYN users where possible, so that a user logging into OZRYN is actually getting a Medplum token behind the scenes (this way, when they interact with data, Medplum's authorization rules apply). You can integrate external SSO (say your organization's Okta) at the Medplum level, so that flows through to OZRYN ³³. The result is a cohesive architecture where Medplum is the **data and auth nucleus**, and your other modules (AI, analytics, UI) orbit around it, drawing from and contributing to the data there.

- **Maintain Compliance and Certifications via Medplum:** One major advantage of using Medplum in OZRYN's architecture is that it carries much of the **compliance burden**. Since Medplum's platform is HIPAA-compliant and ONC-certified out of the box ²⁶ ³¹, OZRYN's developers won't need to custom-build those capabilities. Ensure that wherever possible, data flows and user actions go through Medplum, so that the built-in audit logging and security features apply. For instance, rather than having a separate database where raw genetic data is stored and could be missed in an audit, consider storing genetic results in Medplum (even if as attachments or structured observations). That way, access to them is audited and governed by Medplum's policies. Medplum's ONC certification means it has features like **patient access download** and **data export** (which you can expose if OZRYN needs to allow patients to download their entire record, fulfilling regulations). By aligning with Medplum's capabilities, OZRYN can accelerate meeting regulatory requirements such as those in the 21st Century Cures Act (information blocking rules, etc.) – Medplum's team keeps the platform updated on those fronts (e.g., support for FHIR R4/R5, US Core profiles, etc.). In architecture terms, consider Medplum the **regulated data store** and design OZRYN so that any non-Medplum data stores (like a data lake for analytics) contain either de-identified data or backups, but not the live PHI single source of truth. This approach localizes compliance-sensitive operations in Medplum, which is purpose-built for it.

To conclude, incorporating Medplum as OZRYN's EHR/clinical backend provides a solid, interoperable foundation that **accelerates development and ensures compliance**. Medplum will handle the heavy lifting of data management (standardization, security, auditability, interoperability), allowing the OZRYN team to focus on the “intelligence” layers (analytics, AI, user experience) rather than reinventing EHR fundamentals. By taking advantage of Medplum's out-of-the-box features where possible and extending it via its well-documented APIs and integration hooks, OZRYN can quickly assemble a **unified health data platform** that is both innovative and grounded in proven healthcare architecture. The result will be an architecture where Medplum serves as the **central nervous system for health data** in OZRYN ⁸⁵, and OZRYN's predictive/preventative intelligence modules act as the “brains” that analyze and act on that data – a powerful combination for transformative health insights.

Sources: Medplum Documentation and Guides ⁸⁶ ¹ ² ⁵ ⁶, ¹⁰ ¹⁴ ¹⁷ ⁸⁷ ³¹ ²⁶, etc. (Medplum developer docs, enterprise features, and third-party case studies as cited above).

¹ ² ⁵ ⁶ ⁷ ⁸ ⁵² ⁵³ ⁵⁷ ⁶¹ Building Healthcare Apps with Medplum: All You Need To Know | TechMagic

<https://www.techmagic.co/blog/medplum>

³ ⁵⁶ ⁶³ Medplum

<https://www.medplum.com/>

⁴ ⁴⁵ ⁴⁶ ⁴⁷ Medplum Provider | Medplum

<https://www.medplum.com/docs/provider>

⁹ ¹⁷ ¹⁸ ¹⁹ ²⁰ ²¹ ²² ⁵¹ ⁸² HL7 Interfacing | Medplum

<https://www.medplum.com/docs/integration/hl7-interfacing>

¹⁰ ¹⁵ ⁷⁶ ⁷⁷ Patient Deduplication Architectures | Medplum

<https://www.medplum.com/docs/fhir-datastore/patient-deduplication>

11 12 **Architecture Overview | Medplum**

<https://www.medplum.com/docs/fhir-datastore/patient-deduplication/architecture-overview>

13 14 42 50 **Patient Deduplication | Medplum**

<https://www.medplum.com/blog/patient-deduplication>

16 23 24 29 30 33 34 41 71 78 79 83 87 **Medplum Enterprise | Medplum**

<https://www.medplum.com/enterprise>

25 **Compliance | Medplum**

<https://www.medplum.com/docs/compliance>

26 31 73 85 **Products | Medplum**

<https://www.medplum.com/products>

27 **Log Streaming - Medplum**

<https://www.medplum.com/docs/integration/log-streaming>

28 **Bots in Production - Medplum**

<https://www.medplum.com/docs/bots/bots-in-production>

32 54 55 **Provider Portal | Medplum**

<https://www.medplum.com/solutions/provider-portal>

35 **External Identity Providers - Medplum**

<https://www.medplum.com/docs/auth/methods/external-identity-providers>

36 40 43 **User Management Guide | Medplum**

<https://www.medplum.com/docs/auth/user-management-guide>

37 38 **Invite a user | Medplum**

<https://www.medplum.com/docs/app/invite>

39 48 49 60 62 **Pricing | Medplum**

<https://www.medplum.com/pricing>

44 65 **Medplum App | Medplum**

<https://www.medplum.com/docs/app>

58 59 64 66 67 68 69 70 72 84 **Self-hosting Medplum | Medplum**

<https://www.medplum.com/docs/self-hosting>

74 75 **Terminologies and Coded Values | Medplum**

<https://www.medplum.com/docs/terminology>

80 81 86 **Welcome to Medplum | Medplum**

<https://www.medplum.com/docs>